

GM ECU Simulator - User Manual

User Manual

A guided start for new users

Edition: 2026-05

Contents

About this manual

1. What this software does

- How it pretends to be real hardware

2. The three modes you need to know

- ECU Simulator mode

- DPS Write mode

- Flash Tool Write mode

- DPS Read and Flash Tool Read (planned)

- Which mode should I pick?

3. System requirements and installation

- Prerequisites

- Step-by-step installation

- Removing the registration

4. Tour of the main window

- Titlebar and status pills

- Menu bar

- Mode selector

- ECU sidebar (left) and Selected-ECU inspector (right)

- Live PID grid

- Workspace tabs

- Status bar

5. Quickstart - ECU Simulator

- Step 1 - Pick ECU Simulator mode

- Step 2 - Add a virtual ECU

- Step 3 - Add a PID

- Step 4 - Connect a J2534 host

6. Quickstart - DPS Write

- Step 1 - What you need

- Step 2 - Pick DPS Write mode

- Step 3 - Run the Prime-From-Archive wizard

- Step 4 - Run DPS

7. Quickstart - Flash Tool Write

- Step 1 - Pick Flash Tool Write mode

- Step 2 - Add and configure the ECU

- Step 3 - Run the flash tool

8. Logging and captures

- The Bus log tab (live, in-memory)

The file log (persistent, on disk)

Programming captures

9. Themes

10. Troubleshooting

My J2534 host does not see GM ECU Simulator in its dropdown

"GM ECU Simulator" appears in the host but the connection fails

The Setup window's PID values look frozen

A DPS session fails partway through

The app reports "cancelled" after I clicked Register

11. Reference

The five modes at a glance

Keyboard shortcuts

Filesystem locations

Per-mode config file

Registry keys written by Register.ps1

12. Glossary

About this manual

This manual is written for a brand-new user. You do not need to know what J2534 is, what DPS is, or how an ECU is programmed in the real world. By the end of the first three sections you will have the software installed, registered, and producing traffic against a real tester application on your PC.

How to read this manual:

- **New users:** read Sections 1 to 4 in order, then pick the Quickstart for the mode you actually want to use (Section 5, 6, or 7).
- **Returning users:** the Reference (Section 11) and Troubleshooting (Section 10) sections are the day-to-day pages.
- **Looking for something specific:** the Glossary (Section 12) explains every acronym used in the app and in this document.

A note on the three big concepts

Sections 2 and 3 introduce three terms you will see everywhere in the app: ECU Simulator, DPS Write, and Flash Tool Write. Read those before clicking around - they are the single biggest source of confusion for new users.

1. What this software does

GM ECU Simulator is a Windows application that pretends to be one or more GM vehicle ECUs (the small computers that run an engine, transmission, or other vehicle subsystem). It speaks the same diagnostic protocols a real ECU speaks, over the same plumbing a real diagnostic tool uses, but no vehicle and no physical interface hardware is involved.

In practical terms it lets you:

- Test a diagnostic application against a virtual ECU on the same PC, with no Tactrix, MongoosePro, or any other hardware plugged in.
- Reproduce a programming session end-to-end against the simulator. GM's internal Diagnostic Programming System (DPS) and the popular open-source flash tools both run against this simulator and complete with a success message.
- Capture every byte that a tool sends during a session. The simulator writes a .bin of the data downloaded into each programming session and a CSV log of every CAN frame in both directions, with human-readable annotations.

- Develop and debug your own diagnostic application. Point it at the simulator instead of a real car, iterate quickly, and avoid the wait-burn-replace loop with a real ECU.

How it pretends to be real hardware

Diagnostic applications on Windows connect to vehicle hardware through a standard interface called J2534. A J2534 device (the Tactrix OpenPort, MongoosePro, Bosch MDI, and dozens of others) is essentially a USB-to-CAN dongle with a small native DLL on the PC side that the diagnostic application loads. The application calls 14 functions in that DLL (PassThruOpen, PassThruWriteMsgs, PassThruReadMsgs, etc.) and the dongle does the rest.

GM ECU Simulator ships its own small native DLL with the same 14 functions. When you register the simulator (a one-click step you do once), Windows tells every J2534-aware application on the machine that there is a new device available called GM ECU Simulator. The application loads the simulator's DLL just like it would load a Tactrix DLL, and from then on every read or write the application makes goes to the running simulator instead of to a real vehicle.

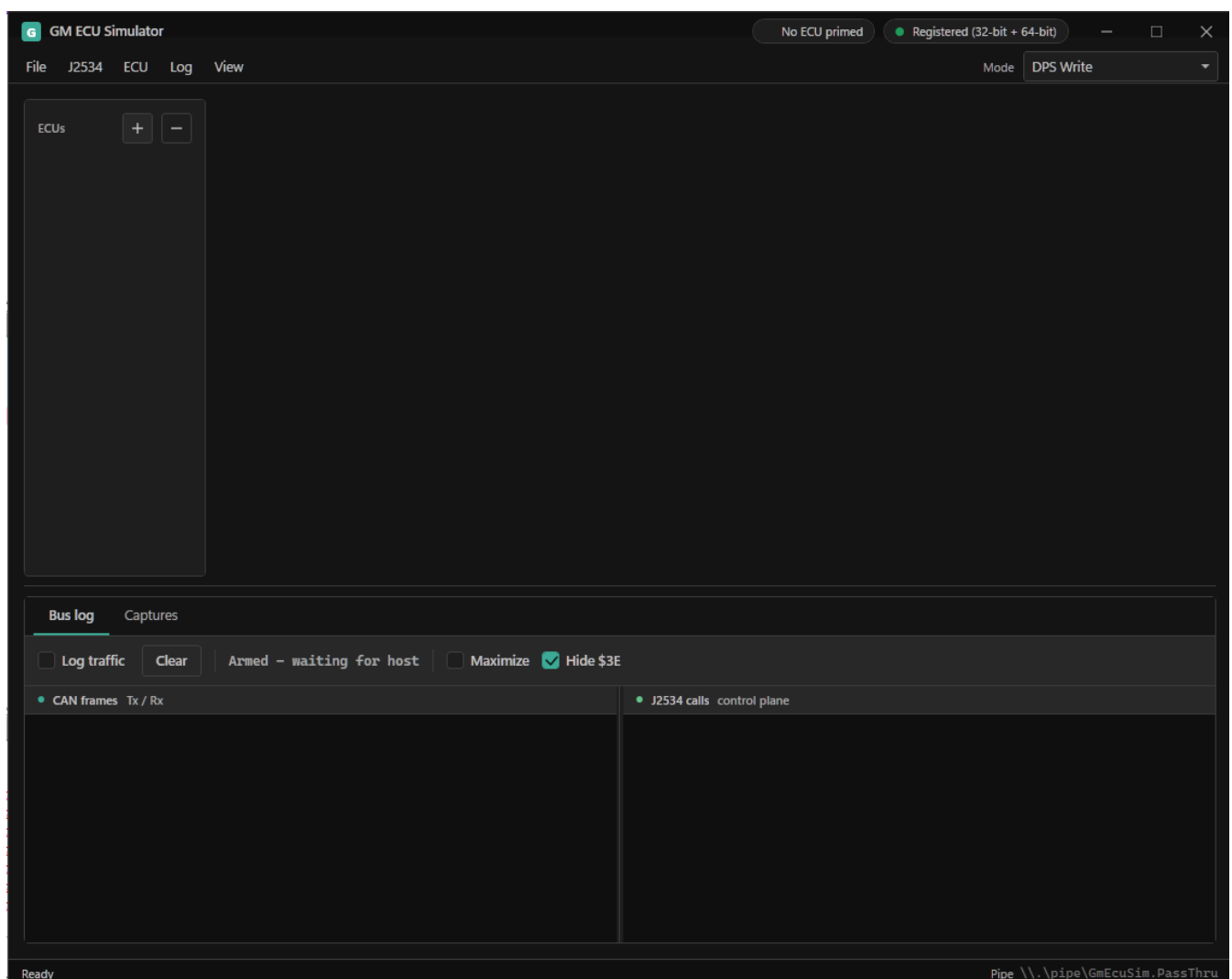


Figure: The main window after installation, with no ECUs configured.

2. The three modes you need to know

The simulator runs in one of five top-level modes, picked from the Mode dropdown in the top-right of the main window. Three of them are fully working today; two are placeholders for planned features.

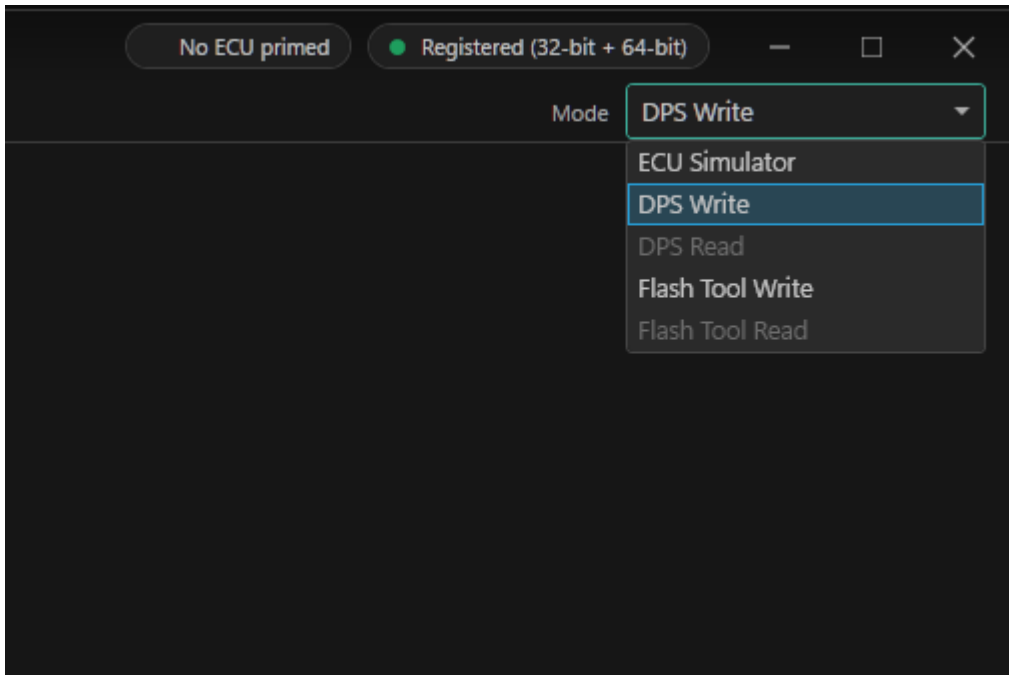


Figure: The Mode dropdown in the top-right corner.

ECU Simulator mode

Use this when: you want to invent one or more virtual ECUs from scratch, define the PIDs they report, and have those PIDs respond with synthesised values (sine waves, square waves, file-streamed data, constants).

This is the original "sandbox" mode. You can have many ECUs at once on the virtual bus, each at its own CAN ID. Each ECU has a list of PIDs with a waveform attached to each PID, so when a tester asks for PID \$1000 the simulator replies with whatever value the waveform is producing right now. State persists to a JSON file in the app's local data folder, so the next time you start the app your ECUs come back.

DPS Write mode

Use this when: you have a real GM DPS programming archive (.zip) for a specific vehicle, and you want the simulator to act as the target ECU that archive was built for, so you can drive a real DPS session against it without touching a real car.

DPS Write mode is single-ECU. The flow is the Prime-From-Archive wizard: you point at a DPS .zip, the simulator parses the utility-file bytecode and the calibration files inside, scans the OS module to extract the PID table, decides which security-access algorithm the archive expects, and builds a virtual ECU that satisfies all of it. You then click File > Save (or let the auto-save remember the path), launch DPS, pick GM ECU Simulator from the J2534 device list, and run the session through to "Programming Successfully Ended".

DPS is GM's internal programming tool

DPS (Diagnostic Programming System) is the dealership-side application GM uses to reflash vehicle modules. A DPS archive is the bundle that defines what a single programming job does: which kernel to upload, which calibration files to flash, which validation routines to run. The archive format is reverse-engineered for the simulator's purposes - you do not need a DPS license to use this mode.

Flash Tool Write mode

Use this when: you want to test the simulator against an open-source flash tool (like 6Speed.T43 for transmissions, or PowerPCM Flasher for engine modules) instead of DPS. The mode is single-ECU and the editor pane stays slimmer.

Flash Tool Write keeps the relevant parts of the inspector (CAN IDs, flow-control settings, security module) visible but hides the PID grid since most flash tools do not read PIDs, only download blocks. The simulator captures the downloaded blocks the same way DPS Write does (see Section 8).

DPS Read and Flash Tool Read (planned)

Both "Read" modes are visible in the dropdown but greyed out. They will eventually let the simulator play back a previously-captured session to a host that thinks it is doing a fresh read. They are not wired up in this release.

Which mode should I pick?

I want to...	Pick this mode
Develop my own scan tool / data logger and need a virtual ECU to read PIDs from	ECU Simulator
Have a real DPS programming session complete successfully on my desk	DPS Write
Run an open-source flash tool against a virtual ECU	Flash Tool Write
Replay a captured programming session back at a host (not yet supported)	DPS Read / Flash Tool Read (planned)

3. System requirements and installation

Prerequisites

- **Operating system:** Windows 10 or Windows 11, 64-bit.
- **.NET 9 Desktop Runtime:** free download from Microsoft. The app's installer will prompt you if it is missing.
- **Visual C++ 2015-2022 Redistributable:** both 32-bit (x86) and 64-bit (x64) versions, because the simulator ships a 32-bit and a 64-bit native shim.
- **Administrator rights for one step:** registering the simulator as a J2534 device writes to the Windows registry under HKLM\SOFTWARE\PassThruSupport.04.04. The app will prompt you (UAC) at exactly the right moment - you do not need to launch the app as Administrator.

Step-by-step installation

1. Unzip the release archive into a folder of your choosing. A typical location is C:\Program Files\GM ECU Simulator, but any folder works - the app does not assume a specific install path.
2. Double-click GmEcuSimulator.exe to launch the app. The main window opens and the J2534 status pill in the top-right will read "Not registered".

3. Open the J2534 menu and click **Register as J2534 device...** Windows will display a UAC prompt because registering writes to HKLM. Click Yes.
4. After the UAC dialog closes the status pill updates within a second or two. You want to see "Registered (32-bit + 64-bit)".
5. Sanity check: open the J2534 menu again and click **Show registered devices...** You should see "GM ECU Simulator" listed under both the 64-bit and 32-bit registry views, each pointing at the matching shim DLL in your install folder.

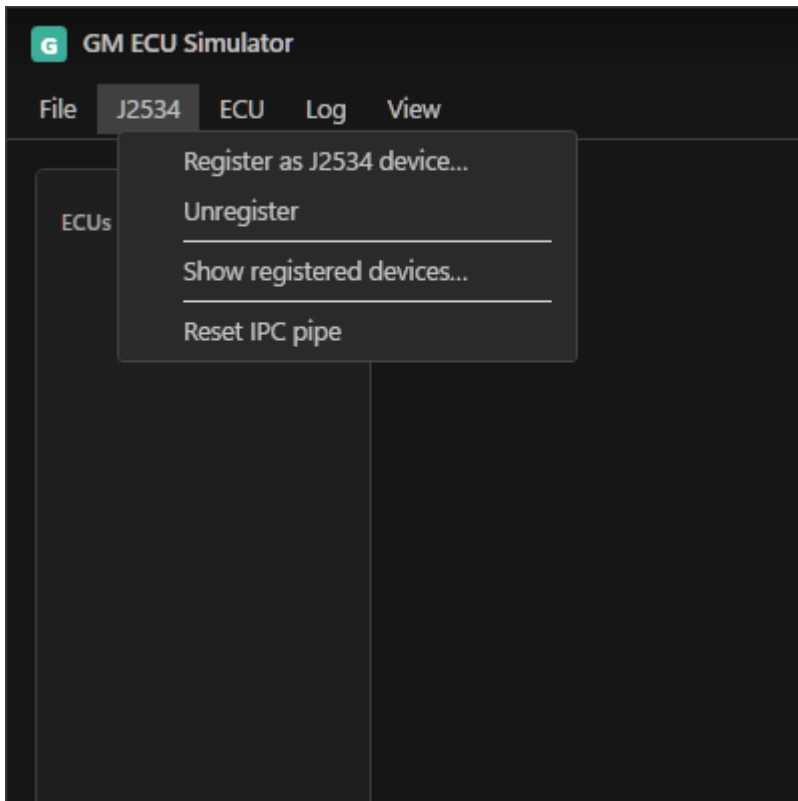


Figure: The J2534 menu open, showing the four registration-related items.

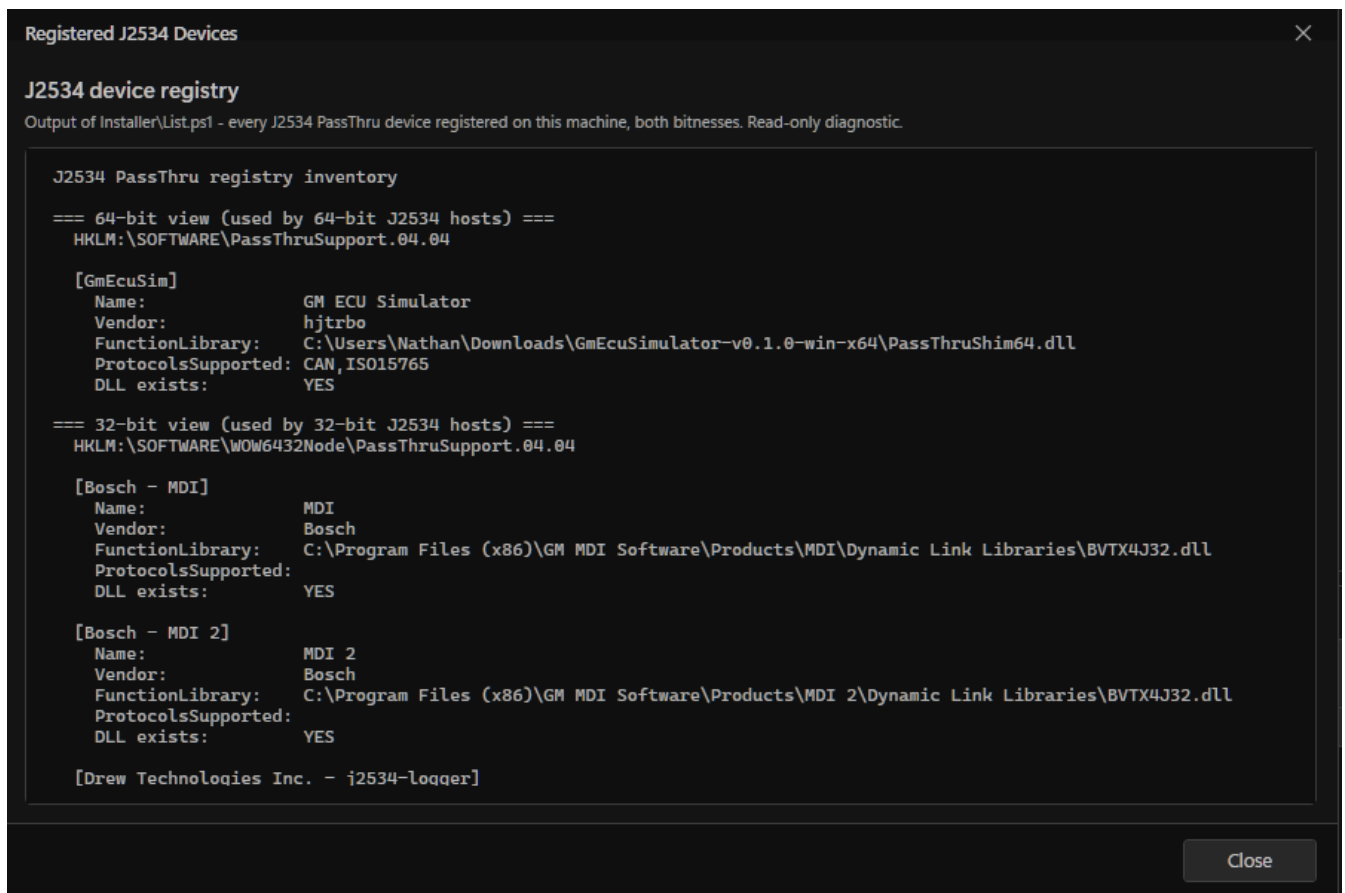


Figure: The Registered Devices window after a successful registration. The simulator appears under both 64-bit and 32-bit views.

Why two bitnesses?

Windows cannot mix bitnesses in a single process: a 32-bit application can only load a 32-bit DLL, and a 64-bit application can only load a 64-bit DLL. Most older diagnostic apps (Tech 2 Win, GDS, Bosch MDI, Tactrix OpenPort, DPS itself) are 32-bit. Newer tools (and the sibling GM DataLogger project) are 64-bit. Registering both bitnesses means the simulator is available to every J2534 host on your PC regardless of the host's architecture.

Removing the registration

Open the J2534 menu and click **Unregister**. This removes only the GM ECU Simulator entries from the registry; no other vendor's J2534 device is touched. The app itself stays installed; you can re-register at any time by clicking Register again.

4. Tour of the main window

The whole app lives in one window. Here is a tour from top to bottom.

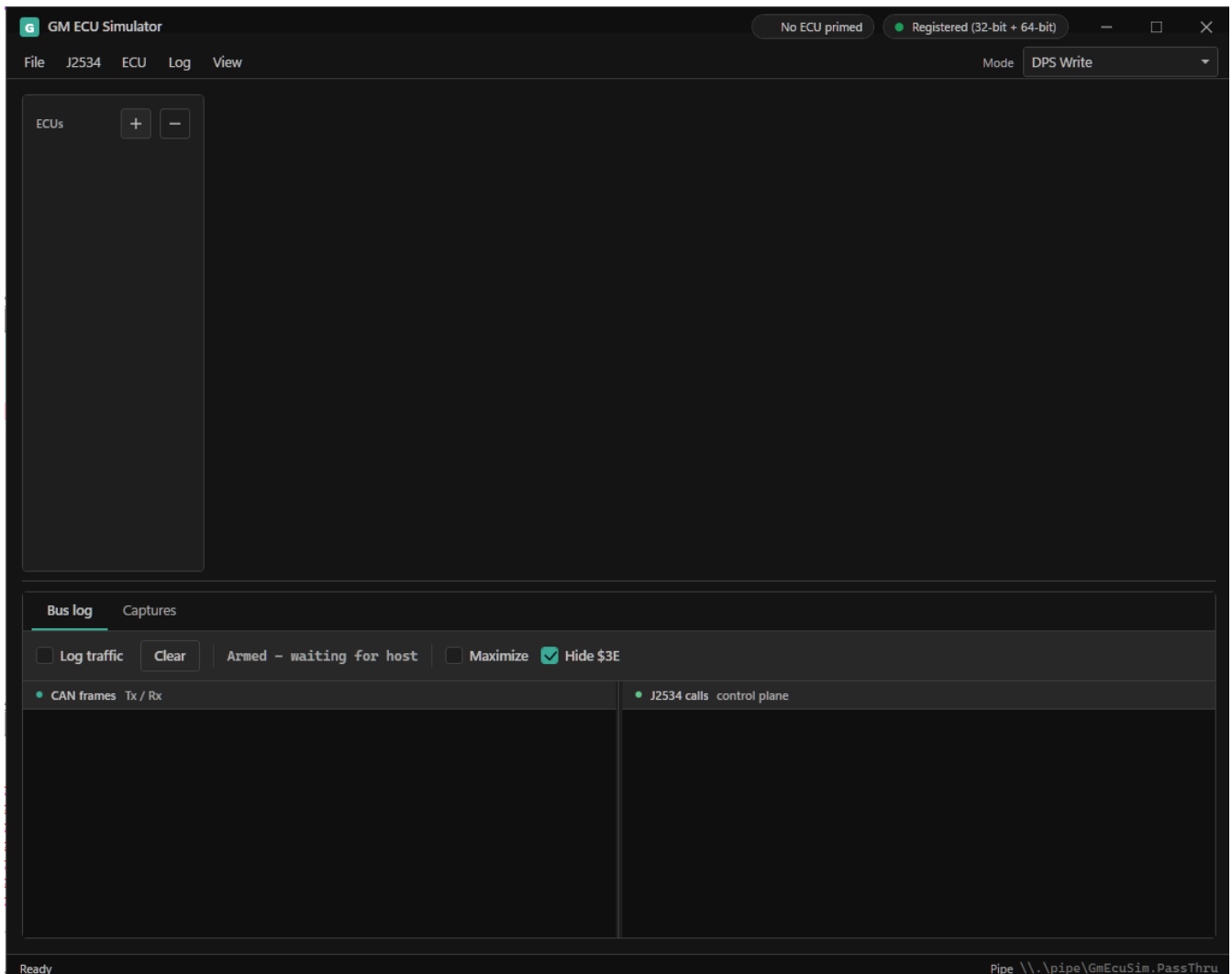


Figure: The main window with the major regions labelled. (Suggested annotation overlay: 1 Titlebar, 2 Status pills, 3 Menu bar, 4 Mode selector, 5 ECU list, 6 Selected-ECU inspector, 7 Live PID grid, 8 Workspace tabs, 9 Status bar.)

Titlebar and status pills

Two pills sit on the right side of the titlebar:

- **Selected-ECU security pill (left):** shows the \$27 SecurityAccess state of the currently-selected ECU - green when unlocked, amber when locked, red when in a failed-attempt lockout. Only appears when an ECU is selected.
- **J2534 registration pill (right):** shows whether the simulator is currently registered with Windows as a J2534 device. Green dot = registered, amber/red = not registered or partially registered.

Menu bar

Six top-level menus. The most commonly-used items:

Menu	Notable items
File	New, Open, Save, Save As, Import, Export, Clear primed archive, Exit. Ctrl+N / Ctrl+O / Ctrl+S work as you would expect.
J2534	Register, Unregister, Show registered devices, Reset IPC pipe (use if a host disconnects messily and the simulator stops accepting new connections).
ECU	Reset state (power-cycle every ECU on the bus), Drive HW \$3E keepalives (toggle whether the simulator runs the tester-present timer on the host's behalf).
Log	Master gate for file logging plus four toggles that decide what reaches the log file (J2534 calls, bus traffic, description tags, collapsed bulk transfers).
View	Open the modeless Setup window (PID editor + waveform editor), pick a Theme.

Mode selector

Top-right of the menu bar. Switching modes wipes the current ECU list and prompts to save first if you have unsaved changes in a persistable mode. Read-only modes (DPS Read / Flash Tool Read) are visible but disabled - they are placeholders for planned features.

ECU sidebar (left) and Selected-ECU inspector (right)

The sidebar lists every ECU on the virtual bus. Click an ECU to select it; its fields appear in the inspector on the right. The Add (+) and Remove (-) buttons at the top of the sidebar are mode-aware:

- **ECU Simulator mode:** Add inserts a blank ECU at the next free OBD-II ID (\$7E0/\$7E8 for the first one, \$7E1/\$7E9 for the second, and so on). You can have as many ECUs as you like.
- **DPS Write or Flash Tool Write mode:** Add opens the Prime-From-Archive wizard (see Section 6). Only one primed ECU is allowed at a time, so Add disables once an ECU is on the bus.

Live PID grid

Below the inspector form in ECU Simulator and DPS Write modes, the live PID grid shows every PID this ECU is configured to respond to, with the current sampled waveform value. The values refresh ten times per second. This grid is read-only; click "Configure PIDs..." to open the Setup window if you want to edit the PID table or the waveforms.

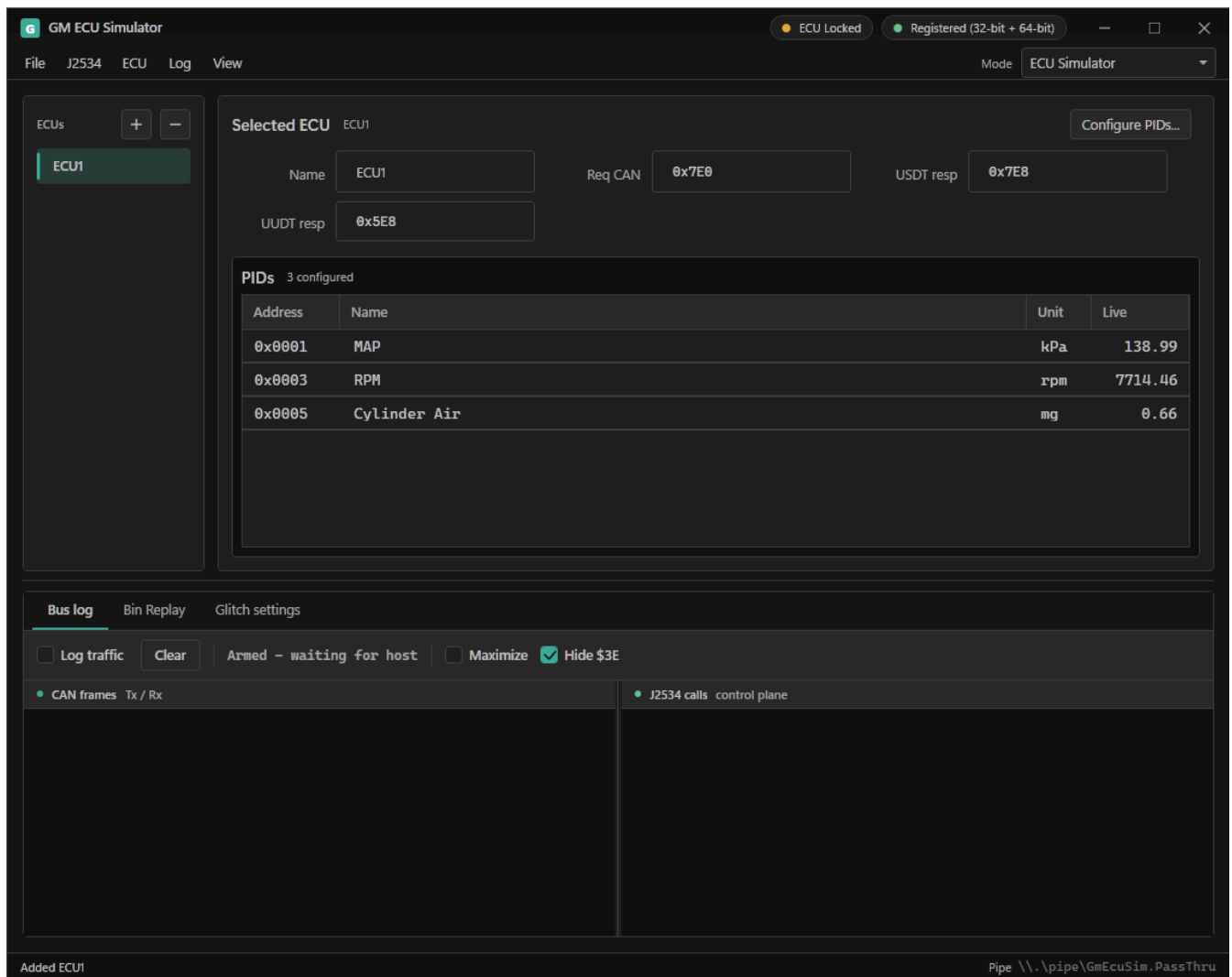


Figure: The Selected ECU inspector with a primed ECU loaded.

Workspace tabs

The bottom half of the window is a tab control. The tabs that are visible depend on the current mode:

- **Bus log:** always visible. Two side-by-side text panes - CAN frames (left) and J2534 calls (right). Use the toolbar checkbox "Log traffic" to start/stop the live capture into the panes; "Hide \$3E" suppresses tester-present keepalives so the log is readable.
- **Bin Replay:** visible in ECU Simulator mode only. Load a .bin produced by the sibling GM DataLogger to replay recorded channel data through the live PID grid.

- **Glitch settings:** visible in ECU Simulator mode only. UI for per-service probability of injected NRCs / dropped frames / corrupted bytes. (Note: defined but not yet wired into the bus logic.)
- **Captures:** visible in DPS Write and Flash Tool Write modes. Browser for the .bin captures the simulator wrote automatically during programming sessions.

Status bar

Bottom of the window. Shows the current status text on the left ("Idle", "Listening", "Connected", error messages), and on the right the primed archive name (DPS modes) plus the named-pipe path the shim DLL forwards to: \\.\pipe\GmEcuSim.PassThru.

5. Quickstart - ECU Simulator

Goal of this section: get a virtual ECU on the bus, configure one PID, and read its value from a J2534 host application.

Step 1 - Pick ECU Simulator mode

1. In the top-right of the menu bar, set Mode to "ECU Simulator". If you are already in another mode the app prompts to save the current configuration first - say Yes or No depending on whether you want to keep it.

Step 2 - Add a virtual ECU

1. Click the + button at the top of the ECUs sidebar. A new ECU appears (default name "ECU1", request CAN ID \$7E0, response CAN ID \$7E8). Click it once to select it.
2. In the inspector, rename it (the Name field) and adjust the CAN IDs if you need to match a specific tester's expectations. Most testers default to \$7E0/\$7E8 and \$5E8, so the default is usually correct.

Step 3 - Add a PID

1. Click **Configure PIDs...** in the inspector header. The Setup window opens.
2. Click + in the PID table to add a row. Set Address to \$1000 (or any 16-bit value), Name to "My Test PID", Size to Word, and a scaling formula such as $\text{raw} * 0.1$ to convert the raw 16-bit value to a real-world unit.
3. In the Waveform panel, pick Sin, set Amplitude to 5000, Frequency to 0.5 Hz, and Offset to 5000. The PID will report values that sweep between 0 and 10000 raw counts, or 0 to 1000 once scaled.
4. Close the Setup window. The Live PID grid in the main window now shows the PID with its current sampled value, refreshing 10 times a second.

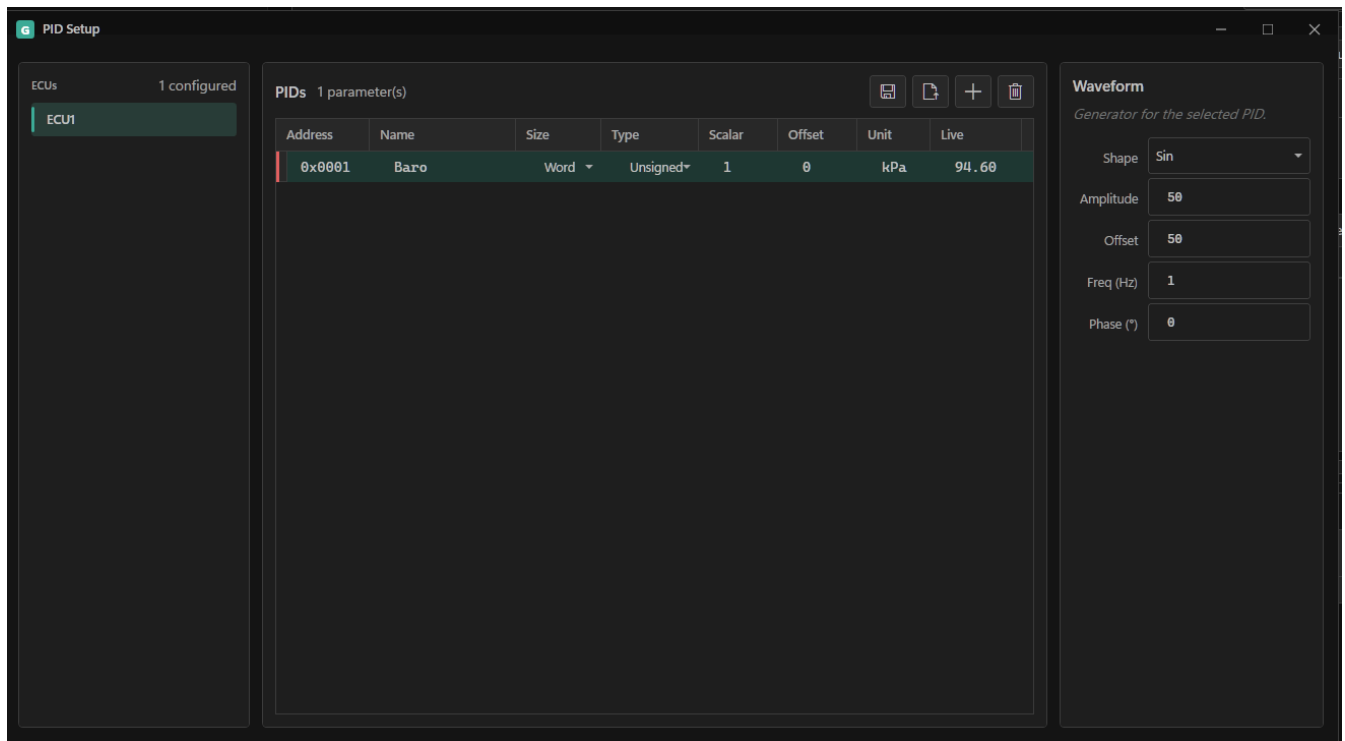


Figure: The Setup window with one PID configured and a sine waveform attached.

Step 4 - Connect a J2534 host

1. Launch any J2534-aware application on your PC. For example, the sibling GM DataLogger project, or any third-party scan tool.
2. Open its device-selection dialog and pick "GM ECU Simulator". The simulator's status bar should update to show a connected client.
3. Have the host read PID \$1000 from the ECU at CAN ID \$7E0. The Bus log tab will show the request frame coming in and the response frame going out, both annotated.

Tip - turn on file logging early

The Log menu's "Log to file" option writes every frame to a CSV in %LOCALAPPDATA%\GmEcuSimulator\logs\bus logs\. Turn it on before you start any serious testing - it is much easier to grep through the CSV than to scroll the live textbox after the fact.

6. Quickstart - DPS Write

Goal of this section: prime a virtual ECU from a real DPS archive, run a real DPS programming session against it, and see the captured downloaded data.

Step 1 - What you need

- A DPS programming archive: a .zip file containing one utility file (the bytecode that drives the session) and one or more calibration files. The simulator has been validated against two such archives as of the current release.
- A working installation of GM DPS on the same PC, or another DPS-compatible programming tool. The simulator only emulates the ECU side; you still need a tool to drive the session.

Step 2 - Pick DPS Write mode

1. In the top-right of the menu bar, set Mode to "DPS Write". The editor pane reconfigures: ECU sidebar shrinks to single-ECU, the Captures tab appears at the bottom.

Step 3 - Run the Prime-From-Archive wizard

1. Click the + button in the ECUs sidebar. The DPS Prime Wizard opens at Page 1: Archive.
2. Click **Browse for archive...** and pick the .zip. The wizard parses it on the spot and shows the archive name, utility file name, calibration file count, and (if present) the OS part number extracted from the OS module header.

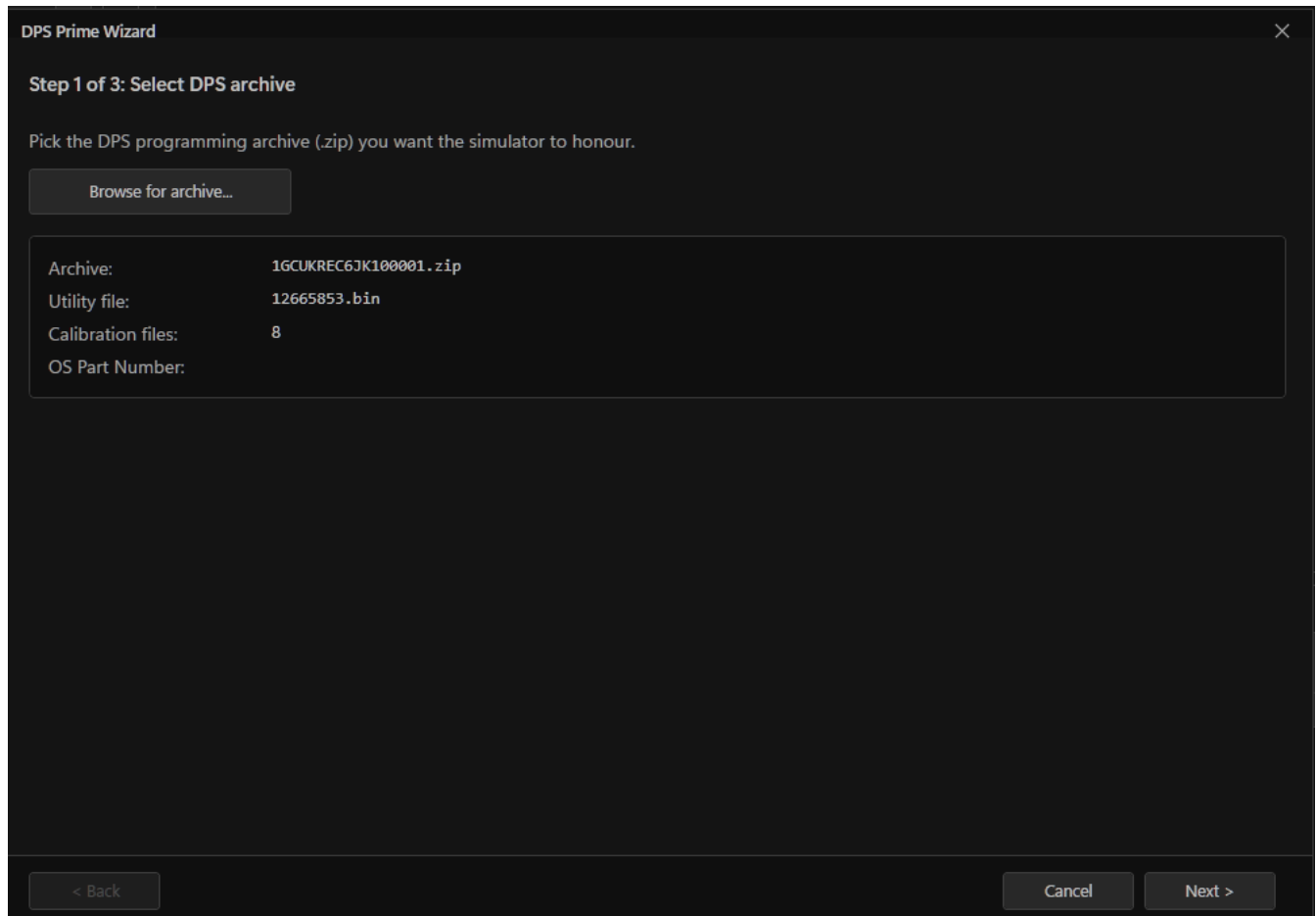


Figure: Prime Wizard - Page 1: archive loaded successfully.

1. Click Next to advance to Page 2: Phase 3 Review. This page shows the post-flash configuration steps the wizard inferred from the utility-file bytecode - which DIDs the tester will write, which PIDs it will read back, which security-access algorithm it will use, and so on.

DPS Prime Wizard

Step 2 of 3: Review Phase 3 reads

DPS will perform these reads after the cal flash completes. The Value column is what the simulator will return for each read. Rows marked "bytecode" are pinned by the archive's \$53 COMPARE_DATA literals - don't change them or DPS will reject the response. Fill empty rows manually, from a bin, or auto-populated.

Load from bin... Auto-populate empty rows

Step	Read	Source	Length	Value (hex, editable)
1	\$22 0x155B	bytecode	1	02
2	\$1A 0x90	(empty)	8	00 00 00 00 00 00 00 00
3	\$22 0x155B	bytecode	1	04
4	\$22 0x155B	bytecode	1	80
5	\$22 0x155B	bytecode	1	00
6	\$1A 0xA0	(empty)	8	00 00 00 00 00 00 00 00
7	\$22 0x2049	bytecode	1	40
8	\$1A 0x90	(empty)	8	00 00 00 00 00 00 00 00
9	\$1A 0xDF	(empty)	4	00 00 00 00

< Back Cancel Next >

Figure: Prime Wizard - Page 2: Phase 3 review.

1. Click Next to advance to Page 3: Commit Summary. This page is the final review before priming. Confirm everything looks right.

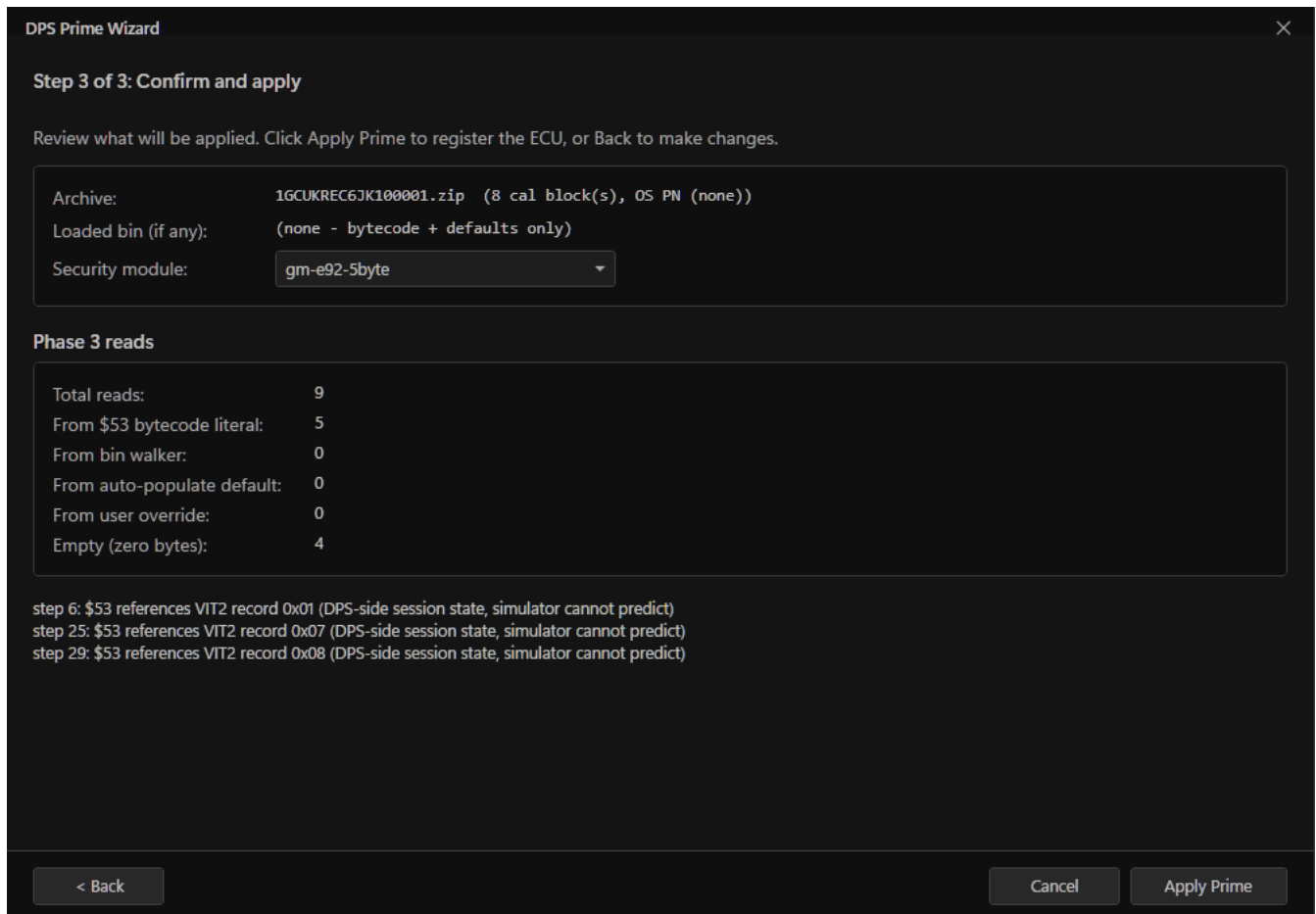


Figure: Prime Wizard - Page 3: commit summary.

1. Click **Apply**. The simulator builds the virtual ECU and registers it at the standard OBD-II CAN IDs. The wizard closes; the new ECU appears in the sidebar; the inspector shows its configuration. A full report of the prime is written to %LOCALAPPDATA%\GmEcuSimulator\logs\death by dps\GmEcuSimulator-prime-<timestamp>.txt - the simulator's own diary of what the archive told it to do.

Step 4 - Run DPS

1. Launch DPS. In its device-selection step, pick "GM ECU Simulator". DPS thinks it is talking to a Tactrix or MongoosePro.
2. Pick the same programming job in DPS as the archive you primed from. Step through the wizard as usual - DPS will run its full \$28 / \$A5 / \$27 / \$34 / \$36 / \$20 / \$AE sequence against the simulator.
3. Watch the Bus log tab. You will see the entire programming flow play out frame by frame, with the calibration data captured to %LOCALAPPDATA%\GmEcuSimulator\logs\captures\.
4. If the session ends with "Programming Successfully Ended" you have a healthy round-trip. If it does not, the prime report and the bus log together should tell you exactly which step failed and what NRC the simulator returned.

Edit the prime without re-running the wizard

Once an ECU is primed you can click "Edit prime..." in the inspector header to re-open the wizard with all of the previous choices pre-filled. The ECU is replaced atomically on Apply, so you cannot end up with half-edited state on the bus.

7. Quickstart - Flash Tool Write

Goal: emulate a programmable ECU for an open-source flash tool such as 6Speed.T43 (transmission control modules) or PowerPCM Flasher (engine control modules).

Step 1 - Pick Flash Tool Write mode

1. Set Mode (top-right) to "Flash Tool Write". The editor pane changes: the live PID grid disappears (most flash tools do not read PIDs), the Flow-Control fields appear in the inspector, and the Captures tab is available at the bottom.

Step 2 - Add and configure the ECU

1. Click + to add an ECU. Unlike DPS Write, this mode does not run a wizard - you configure the ECU manually because the flash tool defines the wire format directly, not through a DPS-style archive.
2. In the inspector, set the request CAN ID and response CAN IDs (USDT for service responses, UUDT for unsolicited messages) to whatever the flash tool expects.
3. **Flow control:** set FC.BS (Block Size) to the value the flash tool requires. 6Speed.T43 specifically requires BS=1 - it sniffs the FC tail of the simulator's reply for the literal pattern "01" to recognise a T43 TCM and will not start the kernel upload otherwise. PowerPCM Flasher is more permissive; BS=0 is fine.
4. **Security module:** pick the seed/key algorithm the target ECU would use. The dropdown lists every algorithm the simulator knows about - gm-t43 for the T43, gm-algo89-2byte / gm-algo92-2byte for PowerPCM-compatible modules, gm-bypass-2byte for an unconditional bypass.

Step 3 - Run the flash tool

1. Launch the flash tool. Pick "GM ECU Simulator" as the J2534 device.
2. Run the download as you normally would. The simulator handles the full \$34 (RequestDownload) / \$36 (TransferData) / \$37 (RequestTransferExit) sequence and writes a capture file to %LOCALAPPDATA%\GmEcuSimulator\logs\captures\ named after the ECU, the timestamp, the base \$36 address, and the byte count.

This is how you reverse-engineer a flash tool

The simulator is permissive - it accepts download flows that a real ECU might reject - so it is well-suited for running an unknown flash tool against it just to learn what the tool does. Turn on file logging with Append description tag and Include J2534 calls, run one session end-to-end, and the resulting CSV is essentially a transcript of the tool's expected wire flow.

8. Logging and captures

Two independent recording mechanisms run side-by-side. They serve different purposes and you can have either or both running at the same time.

The Bus log tab (live, in-memory)

The Bus log tab in the main window shows a rolling buffer of recent frames in two text panes. It is useful for watching things happen as they happen but it caps at a few thousand lines and is not persisted.

- **Log traffic checkbox (toolbar):** master gate. Off by default - in fast streaming scenarios (a \$AA Fast 25Hz DPID stream, for example) the textbox would flood within seconds.
- **Hide \$3E checkbox (toolbar):** suppress TesterPresent keepalives (\$3E requests and \$7E positive responses) from this window only. File log is unaffected.
- **Maximize checkbox (toolbar):** collapse the ECU/inspector pane and let the workspace tabs fill the full window.

The file log (persistent, on disk)

Open the **Log menu**. The first item, **Log to file**, is the master gate. When on, a dedicated background thread streams every frame as CSV to %LOCALAPPDATA%\GmEcuSimulator\logs\bus logs\bus_<timestamp>.csv. The other four items in the menu fine-tune what is captured:

- **Include J2534 calls:** write every pipe-level call (Open, Connect, StartMsgFilter, ReadMsgs, etc.) into the log.
- **Include bus traffic:** write every CAN frame in both directions.
- **Append description tag:** annotate each bus-frame line with a short human-readable tag ("SecurityAccess - RequestSeed", "ISO-TP FF (3110B) - TransferData @ 003FAFE0").

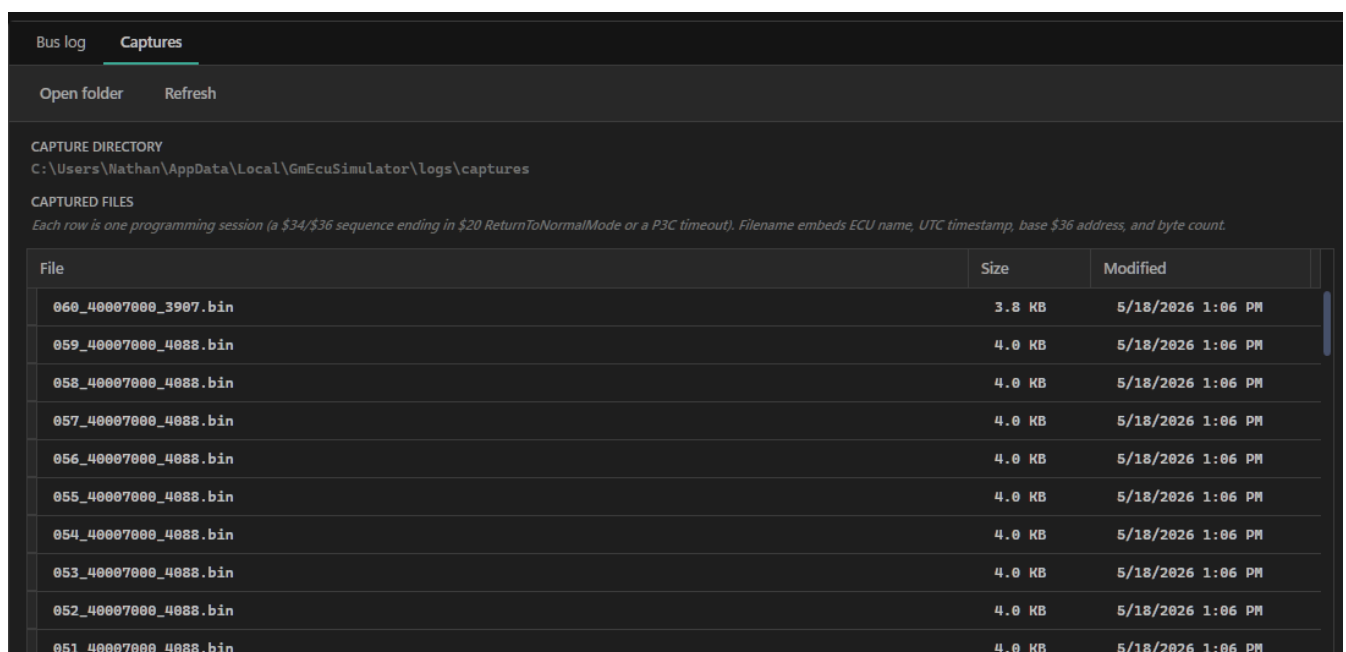
- **Collapse bulk transfers:** during long ISO-TP downloads, show the first three and last three consecutive frames and replace the middle with a single "bulk transfer collapsed: N frames hidden" marker.

Open the log folder: Log > Open folder... opens

%LOCALAPPDATA%\GmEcuSimulator\logs\bus logs\ in Explorer.

Programming captures

Whenever the simulator sees a programming download (\$34 / \$36 / \$37 sequence) it writes a .bin file containing the actual downloaded bytes. These are not opt-in - they are always written - but they only exist for sessions where a download actually happened. View them from the Captures tab (visible in DPS Write and Flash Tool Write modes), or by opening the capture directory directly.



File	Size	Modified
060_40007000_3907.bin	3.8 KB	5/18/2026 1:06 PM
059_40007000_4088.bin	4.0 KB	5/18/2026 1:06 PM
058_40007000_4088.bin	4.0 KB	5/18/2026 1:06 PM
057_40007000_4088.bin	4.0 KB	5/18/2026 1:06 PM
056_40007000_4088.bin	4.0 KB	5/18/2026 1:06 PM
055_40007000_4088.bin	4.0 KB	5/18/2026 1:06 PM
054_40007000_4088.bin	4.0 KB	5/18/2026 1:06 PM
053_40007000_4088.bin	4.0 KB	5/18/2026 1:06 PM
052_40007000_4088.bin	4.0 KB	5/18/2026 1:06 PM
051_40007000_4088.bin	4.0 KB	5/18/2026 1:06 PM

Figure: The Captures tab with one or more recorded sessions.

9. Themes

Open **View > Theme** to pick from the bundled colour palettes. Drop your own .xaml palette file into %APPDATA%\GmEcuSimulator\Palettes\ and it appears in the same submenu after the next app restart. The theme system is shared with the sibling GM Theme Manager project - any palette built for that project loads here without modification.

10. Troubleshooting

My J2534 host does not see GM ECU Simulator in its dropdown

This is almost always one of three things:

1. **Registration is not in place.** Check the J2534 status pill in the top-right - it should read "Registered (32-bit + 64-bit)". If it reads "Not registered" or "32-bit only", run J2534 > Register as J2534 device... again.
2. **Bitness mismatch on a strict host.** If your host is 32-bit and the pill says "64-bit only" (or vice versa), the matching shim DLL did not register. The most common cause is that the install folder is missing one of the two PassThruShim DLLs. Reinstall the release zip in full.
3. **The host enumerates with the v05.00 API.** A handful of newer J2534 hosts only look at the v05.00 device list. The simulator is v04.04 only by design. Most hosts fall back to the v04.04 path when v05.00 returns empty, but a strict one will not. Workaround: pick the device by calling `GetDevice("")` (the v04.04 default-open path) instead of enumerating.

"GM ECU Simulator" appears in the host but the connection fails

Open the J2534 menu and click **Reset IPC pipe**. This stops and restarts the named-pipe server inside the simulator. A previously-connected host that disconnected messily can leave the pipe in a state where new opens fail; restarting the server clears that. No UAC.

The Setup window's PID values look frozen

The Live PID grid refreshes ten times a second from the same waveform engine the bus uses, so a frozen grid means the waveform engine itself is stuck. Try **ECU > Reset state** - this power-cycles every ECU, clearing programming sessions, security unlocks, download buffers, and DPID schedules.

A DPS session fails partway through

Three things to check in order:

1. **Read the prime report.** Every prime writes `%LOCALAPPDATA%\GmEcuSimulator\logs\death by dps\GmEcuSimulator-prime-
<timestamp>.txt` - a step-by-step record of what the wizard inferred from the archive. If the report flags a step it could not infer cleanly, that is almost certainly the failure point.
2. **Read the bus log.** Turn on Log > Log to file with Append description tag before re-running. The last few lines of the CSV before the failure tell you which request the

simulator could not satisfy and which NRC it returned.

3. **Re-prime with edits.** Click Edit prime... in the inspector header, walk through the wizard, and adjust whatever the report flagged. Apply replaces the ECU atomically - no need to clear state first.

The app reports "cancelled" after I clicked Register

That message means you clicked No on the UAC prompt. Click Register again and click Yes when the UAC dialog appears.

11. Reference

The five modes at a glance

Mode	ECU count	State persists?	Config file
ECU Simulator	Many	Yes	ecu_simulator_config.json
DPS Write	One	No (per-run, primed from archive)	dps_write_config.json (path only)
DPS Read (planned)	-	-	dps_read_config.json
Flash Tool Write	One	Yes	flash_write_config.json
Flash Tool Read (planned)	-	-	flash_read_config.json

Keyboard shortcuts

Shortcut	Action
Ctrl+N	File > New
Ctrl+O	File > Open...
Ctrl+S	File > Save
Ctrl+Shift+S	File > Save As...
Ctrl+Shift+P	View > Setup window...

Filesystem locations

All user-writable data lives under %LOCALAPPDATA%\GmEcuSimulator\.

%LOCALAPPDATA%\GmEcuSimulator\

logs\

bus logs\ bus_<timestamp>.csv (file-log output)

captures\ download captures: <ecu>-<ts>-<addr>-<bytes>.bin

death by dps\ GmEcuSimulator-prime-<timestamp>.txt (prime reports)

%APPDATA%\GmEcuSimulator\

Palettes\ drop .xaml palette files here for View > Theme

Per-mode config file

The config file lives in the install folder by default and stores the ECU list, the PID tables, the waveform settings, and any per-mode flags. Use File > Save and File > Open to manage multiple configs side by side.

Registry keys written by Register.ps1

HKLM\SOFTWARE\PassThruSupport.04.04\GmEcuSim (64-bit hosts)

HKLM\SOFTWARE\WOW6432Node\PassThruSupport.04.04\GmEcuSim (32-bit hosts)

Values written into each key:

Name = GM ECU Simulator

Vendor = ECA

FunctionLibrary = full path to PassThruShim64.dll or PassThruShim32.dll

ProtocolsSupported= (DWORD bitmask)

CAN = 1

ISO15765 = 1

12. Glossary

Term	Meaning
CAN	Controller Area Network. The wire protocol every modern vehicle uses for ECU-to-ECU communication. CAN frames carry up to 8 bytes of payload at a CAN ID; the simulator's virtual bus carries the same frames in memory.
CAN ID	An 11-bit (or 29-bit) integer that identifies a CAN frame's sender or intended recipient. GM uses \$7E0-\$7E7 for tester-to-ECU physical requests, \$7E8-\$7EF for ECU-to-tester service responses.
DID	Data Identifier. A 16-bit handle for a piece of identification or configuration data on an ECU (VIN, calibration ID, hardware part number). Read with \$22, written with \$3B.
DPS	Diagnostic Programming System. GM's dealership-side application for reprogramming vehicle ECUs. "DPS Write" mode emulates the target ECU well enough for DPS to complete a real programming job.
DPID	Dynamic Packet Identifier. A streaming message format defined by GMW3110 mode \$AA - the ECU pushes a packed snapshot of N PIDs at a fixed cadence (Slow/Medium/Fast bands).
ECU	Electronic Control Unit. The small computer inside a vehicle module (engine, transmission, body controller). One physical box, one ECU. The simulator emulates any number of ECUs at once.

Term	Meaning
Flash tool	Any application that reprograms an ECU. Includes GM's own DPS but also open-source projects like 6Speed.T43 (transmissions) and PowerPCM Flasher (engine modules). Flash Tool Write mode is tuned for these latter tools.
GMW3110	GM's diagnostic protocol specification ("GMW3110-2010" is the 2010 revision). Defines the modes \$22, \$2C, \$2D, \$AA, \$27, \$34, \$36, \$37, \$3E, etc., that the simulator implements.
ISO-TP	ISO 15765-2, the transport layer that segments diagnostic messages larger than 7 bytes across multiple CAN frames. The simulator implements full reassembly with flow control.
J2534	SAE J2534, the Windows-side standard for vehicle diagnostic interfaces. Every J2534 device exposes the same 14 functions (PassThruOpen, PassThruWriteMsgs, etc.) so any J2534-aware host can use any J2534 device interchangeably.
NRC	Negative Response Code. A one-byte error code an ECU sends when it cannot satisfy a request. Common ones: \$11 serviceNotSupported, \$22 conditionsNotCorrect, \$33 securityAccessDenied, \$78 busyResponsePending.
PassThru	The function prefix used by every J2534 entry point (PassThruOpen, PassThruWriteMsgs, etc.). "PassThru" and "J2534" are used almost interchangeably in this manual.
PID	Parameter Identifier. A 16-bit handle for a sensor value or other live datum (Engine RPM, Coolant Temp). Read with \$22. In ECU Simulator mode you define your own PIDs and attach a waveform to each one.

Term	Meaning
Prime / Prime-From-Archive	The Prime Wizard's job is to take a DPS programming archive and configure a virtual ECU that will satisfy whatever programming flow that archive expects. "Primed" means an ECU has been built from an archive (as opposed to from scratch).
SecurityAccess (\$27)	The mode an ECU uses to challenge a tester before allowing programming. Tester asks for a seed, ECU returns a random number, tester computes a key from the seed, ECU validates the key. Different ECU families use different seed-to-key algorithms; the simulator ships several.
SPS	Service Programming System. Older name for DPS; the bytecode inside a DPS archive is still officially called the SPS Interpreter bytecode.
Shim	The small native DLL (PassThruShim32.dll / PassThruShim64.dll) that Windows registers as the J2534 device. The shim forwards every PassThru call over a named pipe to the running .NET simulator process.